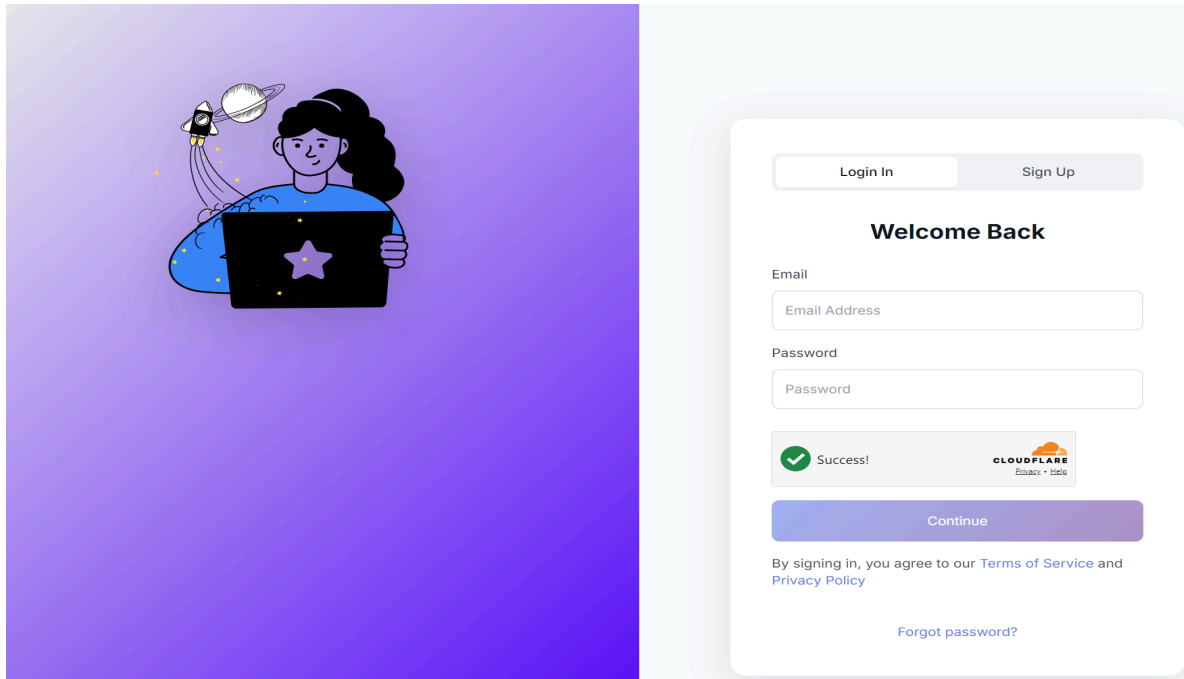


Performance Results :

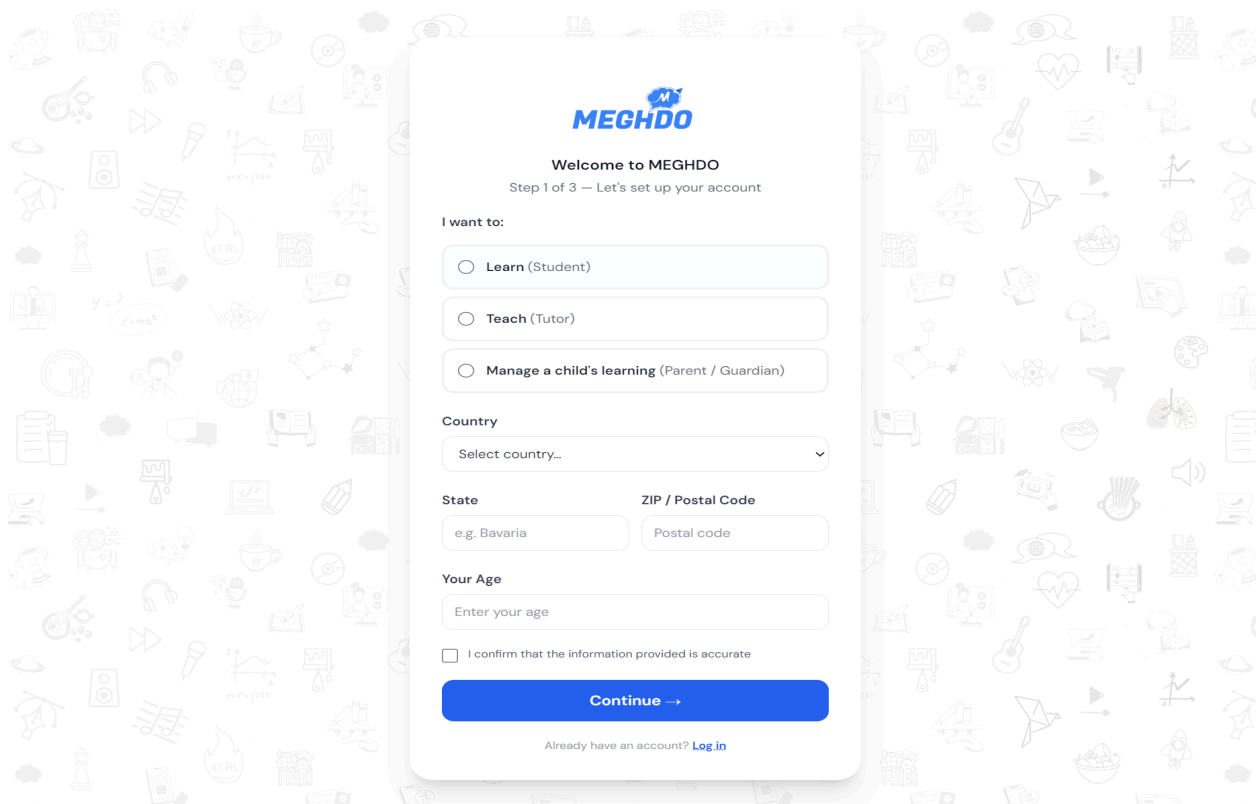
1] sat.meghdo.com :

Page 1 : <https://sat.meghdo.com/login>



- Resource Load Delay (1,591 ms): The image isn't queued until 2,151 ms into the page load. This is because the image is discovered via vendor-evk5PToZ.js rather than being declared in the initial HTML.
- Resource Load Duration (647 ms): The image file is large and is being fetched with a "Low" initial priority.
- CSS Chaining: Multiple Google Fonts stylesheets (css2?family=Inter..., css2?family=Fraunces...) and a local stylesheet (index-B1X112b2.css) are render-blocking.
- Synchronous JavaScript: The script env-config.js is also marked as render-blocking.
- Main Thread Execution (108 ms): A substantial delay occurs within index-CZbBWt_M.js during v8.evaluateModule. This is linked to module assessment and app-setup ProfileCalls.
- Third-Party Scripting (61 ms): External code originating from Cloudflare (main.js) adds to processing congestion on the primary thread.

Page 2 : <https://sat.meghdo.com/onboarding>



The LCP element is a large background image (2400X2400%20BG_Revised-BH3HaXOf.png). The breakdown reveals that the Resource Load Duration accounts for 68% (5,469 ms) of the total LCP time.

- Root Cause: The LCP image is a high-resolution PNG file (2400x2400) served with a "Low" initial priority. It takes over 5 seconds to download on a Fast 4G connection.
- Discovery Delay: There is a 1,655 ms Load Delay before the image even starts downloading. This is likely because the image is discovered via JavaScript or CSS rather than being present in the initial HTML.
- Render Delay: After the image finishes downloading, there is a 579 ms Render Delay. This is caused by long-running main thread tasks that block the browser from painting the pixels. Specifically, a long task (RunTask) (256 ms) and a subsequent long task (RunTask) (265 ms) occur right before the LCP paint.

The primary execution thread encounters substantial bottlenecks throughout the initialization phase, resulting in numerous processing tasks that exceed a 200 ms threshold.

- Intensive Scripting: Significant primary thread overhead is attributed to `Is (ProfileCall)` within the vendor-specific bundle, primarily involving React reconciliation operations.
- External Script Influence: Integration of the Stripe library (`stripe.js`) triggers extended execution sequences (`e (ProfileCall)`) during the vital loading period, consuming 80 ms of processing capacity.
- Pre-paint & Commit Overhead: Substantial durations for `PrePaint` (253 ms) and `Commit` (192 ms) suggest the browser is managing intricate DOM architectures or style calculations prior to visual output.

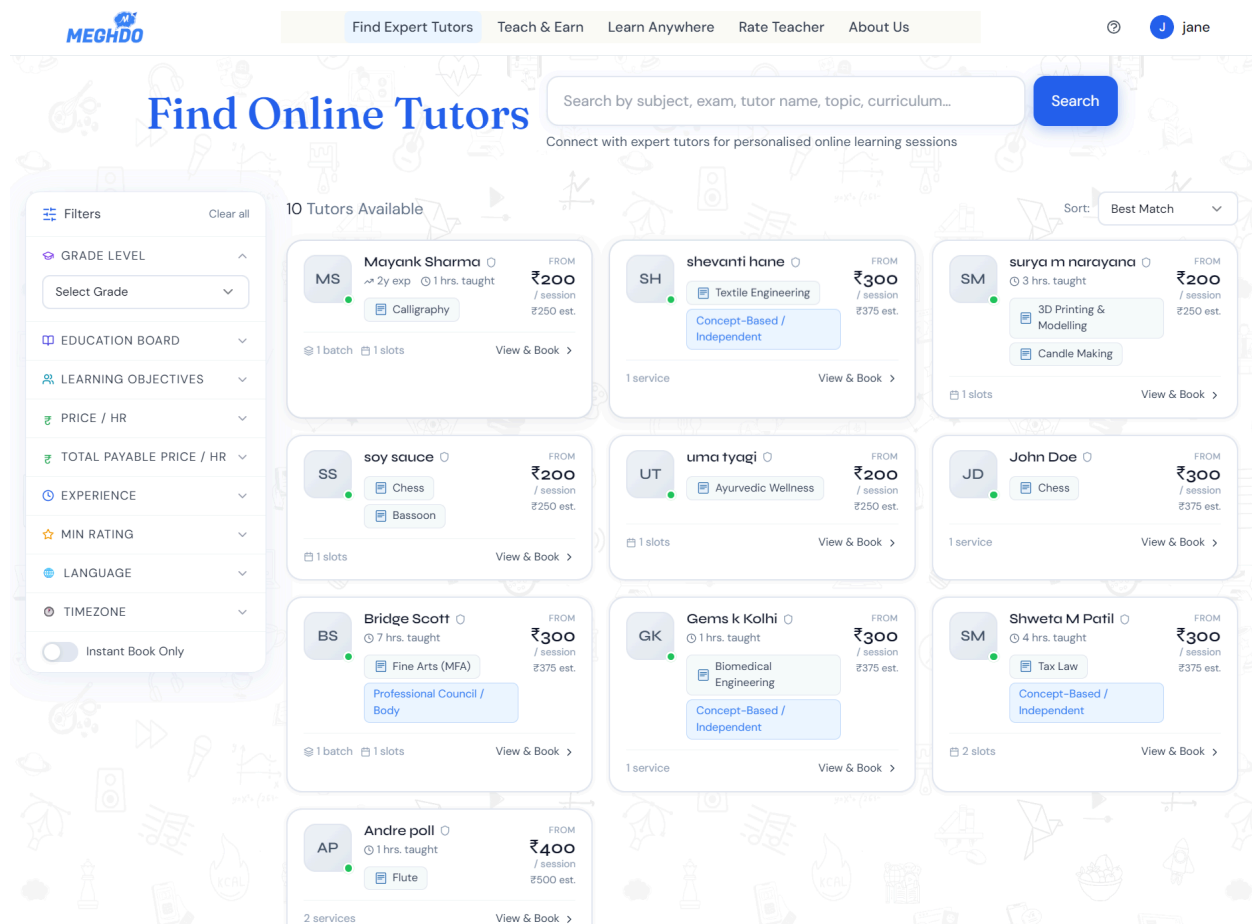


The LCP element is a large background image (Bg_Landing-Page-k8NwtUI8.jpg). The breakdown reveals major bottlenecks in discovery and rendering:

- **Discovery Delay (2,793 ms):** The browser does not discover the LCP image until nearly 3 seconds into the page load. This usually happens because the image is referenced in a CSS file or injected via JavaScript rather than being in the initial HTML.
- **Resource Load Duration (3,536 ms):** The image itself is large and takes over 3.5 seconds to download, even with a "High" final priority.
- **Element Render Delay (2,937 ms):** This is a critical issue. Even after the image is fully downloaded at 7,129 ms, it isn't painted until 10,066 ms. This 3-second gap is caused by the main thread being occupied with other tasks, preventing the browser from rendering the pixels.

The main thread is heavily congested, which directly contributes to the 2.9-second Render Delay.

- **Script Execution:** A significant amount of time (over 1,100 ms) is spent in general "Tasks," with substantial activity from index-CZbBWt_M.js.
- **Third Parties:** Third-party scripts are contributing to the load. Stripe (stripe.js) accounts for 83 ms of main thread time, and Cloudflare scripts are also active during critical phases.
- **Long Tasks:** There are several long tasks exceeding 100 ms, including a 201 ms task (RunTask) and a 193 ms task (RunTask), which block the browser's ability to respond to the user or paint the page.



The LCP element is a large background image (2400X2400%20BG_Revised-BH3HaXOf.png). Its loading process accounts for nearly 70% of the total delay.

- **Discovery Delay (1,443 ms):** The browser does not discover this image until almost 1.5 seconds into the page load. This usually happens when an image is injected by JavaScript or defined in an external CSS file rather than being present in the initial HTML.
- **Massive Download (5,483 ms):** The image is a large PNG being downloaded over a 4G connection. Its initial priority was "Low," which further delayed its acquisition.
- **Render Delay (564 ms):** Even after the image finished downloading at 7,330 ms, the browser couldn't paint it until 7,894 ms because the main thread was busy with heavy JavaScript execution.

The main thread is significantly blocked by long-running JavaScript tasks, which delays rendering and prevents the page from being responsive to user input.

- **React Reconciliation:** A significant amount of time is spent in the `Is (ProfileCall)` function within `vendor-eVv5PToZ.js`. This is the React "work loop," indicating that the initial component mounting and rendering process is very heavy.
- **Long Tasks:** There are several tasks exceeding 250 ms, including a 301 ms task (`RunTask`) and a 282 ms task (`RunTask`). These are large enough to cause noticeable lag.
- **Third-Party Execution:** Stripe's library (`stripe.js`) and Google Fonts are active during the critical path, contributing to the overall script execution time.



Find Expert TutorsTeach & EarnLearn AnywhereRate TeacherAbout Us

🔍 jane

Teach what you know. Build your reputation. Earn globally.

Teach students across India and beyond through structured one-on-one sessions, batches, workshops, and live learning experiences.

[Start Teaching on MEGHDO](#)[Need help deciding what to teach?](#)

Why educators choose MEGHDO

Flexible teaching Teach part-time or full-time on your own schedule.	Global students Reach learners across India, the United States and other countries.	Transparent earnings Set your own session pricing and receive clear payouts.	Professional reputation Build your UTAR reputation through structured student feedback.
--	---	--	---

The LCP element is a text heading (H1). Since it is text and doesn't require a network request, its timing is entirely dependent on when the browser can finish its initial rendering tasks.

- Root Cause: The rendering of the H1 is delayed by render-blocking resources and heavy JavaScript execution.
- Render-Blocking Impact: Five resources, including Google Fonts stylesheets (s-1978, s-1973, s-1958) and the main application CSS (s-1994), block the first paint.
- Main Thread Contention: A long task (RunTask) (95 ms) occurs during the critical loading window. This task involves evaluating the index-CZbBWt_M.js module and executing React's initial "work loop" (ls). Inside this process, a specific function call to Intl.DateTimeFormat() adds about 15 ms of overhead just to determine the user's timezone.

The trace shows a Layout (Layout) event taking 58 ms, which is relatively high for a single pass and suggests a complex DOM structure.

- Observation: The main thread spends 86 ms in total on Layout and 27 ms on Style Recalculation. While these aren't currently causing a "Poor" LCP, they contribute to the high Element Render Delay.

MEGHDO

Find Expert Tutors Teach & Earn Learn Anywhere Rate Teacher About Us

ONLINE TUTORS · INDIA & USA

Find tutors based on real student feedback. Not marketing claims.

Search tutors by subject, exam, budget, teaching style, and UTAR ratings. Book one-on-one online sessions without expensive coaching bundles or long-term lock-ins.

Find Expert Tutors → How UTAR Works

TRANSPARENT TUTOR DISCOVERY
Find the right tutor
Compare tutors intelligently

Maths Tutor Science Coding JEE / NEET SAT P Languages

- Real student feedback
- Pay per session only
- Flexible scheduling
- India & USA tutors

The page's slow start is caused by multiple "render-blocking" resources that prevent anything from appearing on screen for over 6 seconds.

- **Render-Blocking Assets:** Five critical files, including several Google Font stylesheets and the main application CSS (index-B1X112b2.css), must be fully downloaded before the browser can start drawing the page. One font request alone takes over 1.4 seconds.
- **Massive JavaScript Bundles:** The main logic file (index-CZbBWt_M.js) is nearly 800 KB, with about 600 KB of that being unused code for the initial load. This large file forces the browser to spend 5.2 seconds on "main-thread work," essentially locking up the page while it processes code.
- **Inefficient Images:** The logo (Meghdo Logo_Final.png) is a massive 8000x4500px image being scaled down to a tiny 256x144px area in the footer. This unnecessarily consumes 350 KB of bandwidth.

MEGHDO

Find Expert Tutors Teach & Earn Learn Anywhere Rate Teacher About Us

jane

STUDENT-POWERED RATINGS

Rate Your Teacher Anonymously

Discover quality educators with real student reviews. Search by name, city, subject, or institute — and share honest feedback that actually helps.

- 100% Anonymous
- UTAR Verified
- AI-Matched Results
- Free Forever

UTAR* Score CTS Rating

Don't see your teacher?
Add them to the directory and help fellow students share honest feedback.

+ Add Your Teacher

HYISCS • CHEMISTRY • BIOLOGY • ENGLISH LITERATURE • HISTORY • MUSIC THEORY • MARATHI • TELUGU • HINDI • ECONOMICS • COMPUTER SCIENCE • GEOGRAPHY • SANSKRIT • ACCOUNTS

NAME: Teacher name... COUNTRY: IN CITY: City... INSTITUTE: Search institute... SUBJECT: Subject... Q Search

16 results

How ratings work

UTAR UTAR helps students quickly compare and select tutors who consistently deliver high-quality learning experiences.

CTS CTS provides a comprehensive view of a tutor's performance, helping students make informed decisions based

MS Mayank Sharma FROM ₹200 / session ₹250 est. ~2y exp 1 hrs. taught Calligraphy 1 batch 1 slots **Rate** View & Book

SH shevanti hane FROM ₹300 / session ₹375 est. Textile Engineering Concept-Based / Independent 1 service **Rate** View & Book

SM surya m narayana FROM ₹200 / session ₹250 est. 3 hrs. taught 3D Printing & Modelling Candle Making 1 slots **Rate** View & Book

SS soy sauce FROM ₹200 / session ₹250 est. Chess Bassoon 1 slots **Rate** View & Book

UT uma tyagi FROM ₹200 / session ₹250 est. Ayurvedic Wellness 1 slots **Rate** View & Book

GK Gems k Kolhi FROM ₹300 / session ₹375 est. 1 hrs. taught Biomedical Engineering Concept-Based / Independent **Rate** View & Book

- **Excessive Main-Thread Work (11.3s):** The browser's main thread is overwhelmed for over 11 seconds. This is largely caused by JavaScript execution. The main application bundle (index-CZbBWt_M.js) is nearly 800 KB, and the browser spends significant time evaluating and executing this script before the page becomes interactive.
- **Render-Blocking Resources:** Several resources, including Google Fonts and the main CSS file, block the browser from painting any content. One font request takes over 1.2 seconds, delaying the initial render.
- **Large Image Payloads:** The logo (Meghdo Logo_Final.png) is a massive 8000x4500px image file (350 KB) that is being scaled down to a tiny footer icon. This wastes bandwidth and processing power. Additionally, search result avatars are served as unoptimized images.
- **Unused JavaScript:** Approximately 620 KB of the loaded JavaScript is not used for the initial search results page. This code is being downloaded and parsed unnecessarily, delaying the "Time to Interactive."



Find Expert TutorsTeach & EarnLearn AnywhereRate TeacherAbout Us

  jane

Building the Infrastructure for Borderless Learning

Sometimes learning doesn't fail because talent is missing.

It fails because help isn't available at the moment it's needed.

MEGHDO is a structured, on-demand global learning marketplace built to unlock human learning and earning potential.

We believe learning should not be restricted by income, geography, credentials, or timing. Access to help should not depend on where someone lives or how much they can commit upfront, or whether they fit into traditional institutional pathways.

Across the world, learning demand is growing — but access remains fragmented. Many capable people want to teach. Many learners need help. Structural barriers keep them apart. Students often pay for rigid bundles they don't fully use. Progress tracking lacks transparency. Marketplaces struggle with trust, liquidity, and fair economics.

Learning is global. Infrastructure is not.

MEGHDO exists to solve this structural inefficiency.

We are building a marketplace where:

Anyone seeking help can access trusted, flexible learning.

The initial render is blocked by several critical resources that prevent any content from appearing for nearly 1.1 seconds.

- **CSS & Config:** The main stylesheet `index-B1X112b2.css` takes 550 ms to download. Simultaneously, `env-config.js` and three Google Font stylesheets (`s-2285`, `s-2280`, `s-2284`) are all marked as render-blocking.
- **Font Delays:** Even after the stylesheets load, the actual font files (e.g., `Fraunces` and `Inter`) take an additional ~450 ms to download.

The browser's main thread is heavily congested, which is the primary cause of the 2,350 ms render delay.

- **Heavy Script Evaluation:** The evaluation of `index-CZbBWt_M.js` triggers a long task (`RunTask`) of 105 ms. During this, an anonymous function (`p-7352-34812-3374-20 (ProfileCall)`) spends 65 ms on the self-time, involving complex object manipulations and regex matching for configuration data.
- **React Reconciliation:** React's internal "work loop" (`ls`) takes 58 ms in one instance (`p-7352-34812-3557-198 (ProfileCall)`). The function `bs` (`p-7352-34812-3558-201 (ProfileCall)`) is a major contributor, indicating that the initial component mounting is expensive.
- **Expensive Layouts:** There are several significant Layout events, including one lasting 104 ms (`r-11385 (Layout)`) and another at 31 ms (`r-14760 (Layout)`). This suggests a complex DOM structure that the browser struggles to calculate.

MEGHDO Find Expert Tutors Teach & Earn Learn Anywhere Rate Teacher About Us ? J jane

Teach. Impact. Earn

Become a Top-Rated Educator on Meghdo

Join a community of expert tutors. Empower students with your skills while we handle the technical side of your teaching marketplace.

Identity Personal Details
Professional Credentials
Subjects What you teach
Persona Bio & Media
Policies Agreements
Complete Finished

Step 1 of 6 0% Complete

Personal Information

First Name * Middle Name Last Name *

jane doe Last Name

Date of Birth * Gender

dd-mm-yyyy Select Gender

Tutors must be 18+ years of age to teach.

Phone Number *

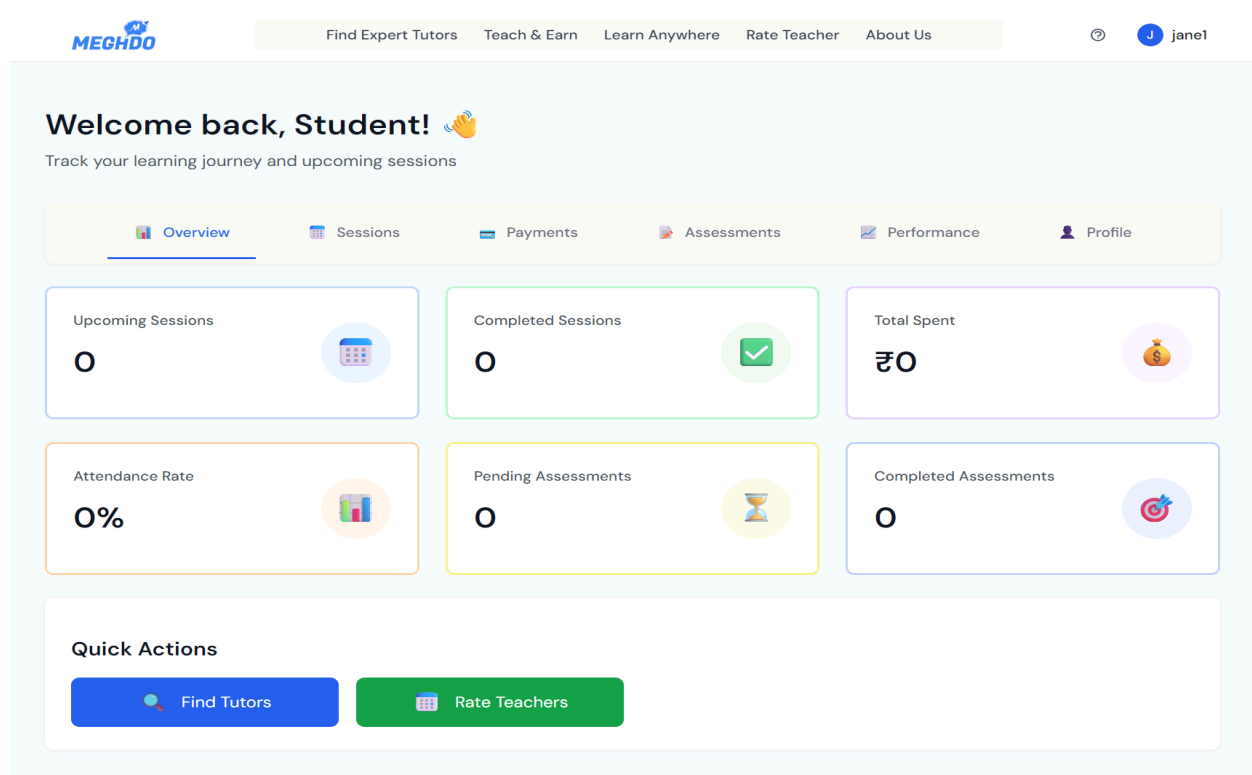
This information is collected for notifications and alerts about your schedule. It will not be shared with any students by the platform to ensure your privacy.

+91 (IN) Phone Number

- Heavy Scripting & React Work: Evaluation of the main bundle index-CZbBWt_M.js triggers significant activity. Specifically, React's internal "work loop" (ls) in vendor-eVk5PToZ.js is a major bottleneck, consuming over 52 ms in a single execution (ls (ProfileCall)).
- Long Tasks: Several tasks are blocking the main thread, including a 221 ms task (RunTask) and a 115 ms task (RunTask). These tasks prevent the browser from painting the LCP element and responding to user input.
- Layout and Style Recalculation: The page spends 444 ms on Recalculate Style and 124 ms on Layout. A significant Layout (Layout) event taking 69 ms indicates a complex DOM structure that is expensive to render. There are also multiple forced reflows (UpdateLayoutTree) early in the load, where JavaScript triggers layout calculations prematurely.

Five resources are marked as render-blocking, preventing the first paint for nearly 1 second.

- Critical CSS & Fonts: The main stylesheet index-B1X112b2.css and several Google Font stylesheets (s-2575, s-2580, s-2579) must be fully downloaded before any content appears.
- Third-Party Impact: Stripe's controller script (stripe.js) and execution (e (ProfileCall)) contribute 83 ms of main thread time during the load.



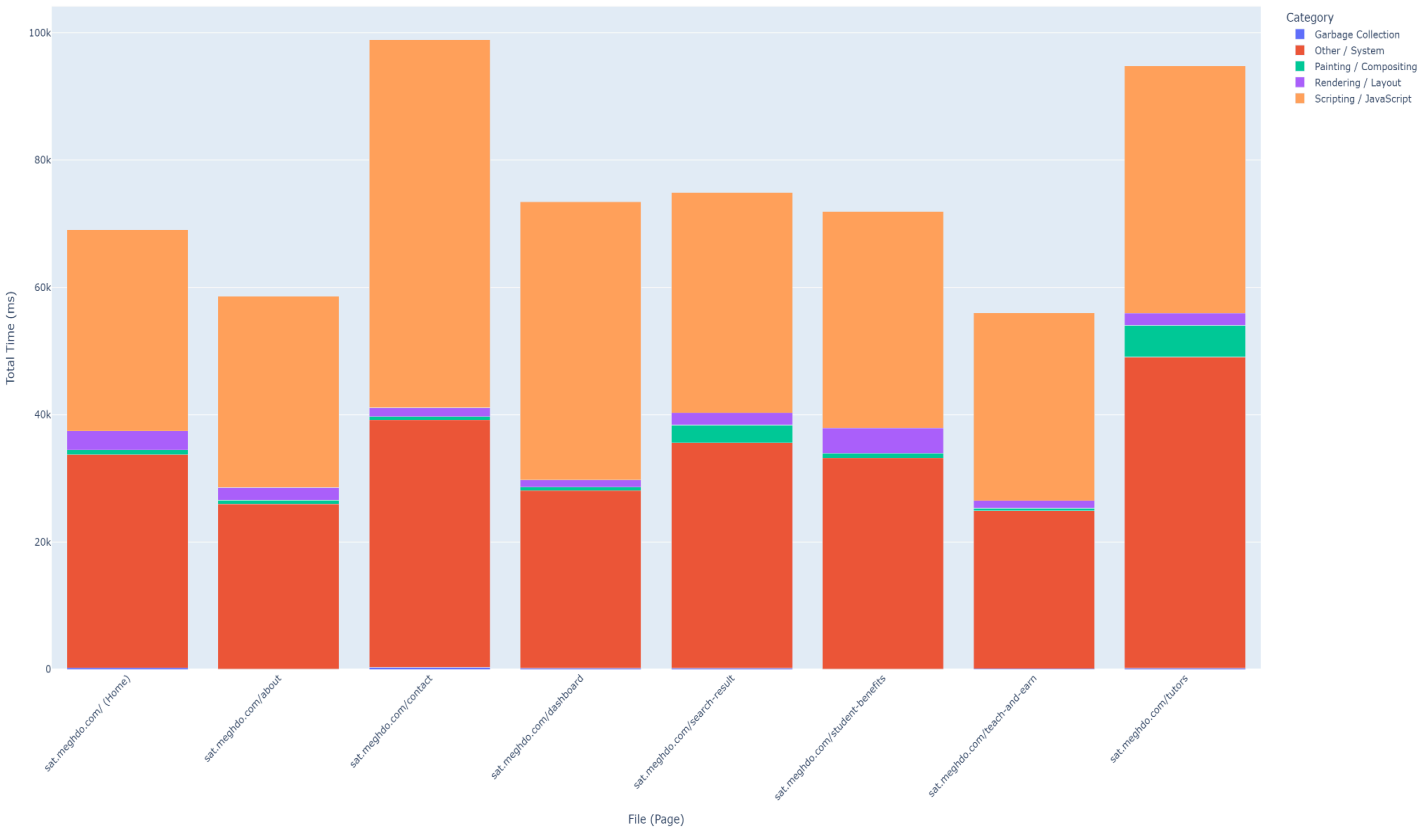
The primary cause of the high LCP is a heavily congested main thread during the critical loading window. The browser is overwhelmed by JavaScript execution and complex layout tasks.

- Heavy Scripting & React Work: Evaluation of the main bundle index-CZbBWt_M.js triggers significant activity. Specifically, React's internal "work loop" (ls) in vendor-eVvk5PToZ.js is a major bottleneck, consuming over 52 ms in a single execution (ls).
- Long Tasks: Several tasks are blocking the main thread, including a 221 ms task (RunTask) and a 115 ms task (UpdateLayer). These tasks prevent the browser from painting the LCP element and responding to user input.
- Layout and Style Recalculation: The page spends 444 ms on Recalculate Style and 124 ms on Layout. A significant Layout (V8.GC_HEAP_EPILOGUE_SAFEPOINT) event taking 69 ms indicates a complex DOM structure that is expensive to render. There are also multiple forced reflows (RunTask) early in the load, where JavaScript triggers layout calculations prematurely.

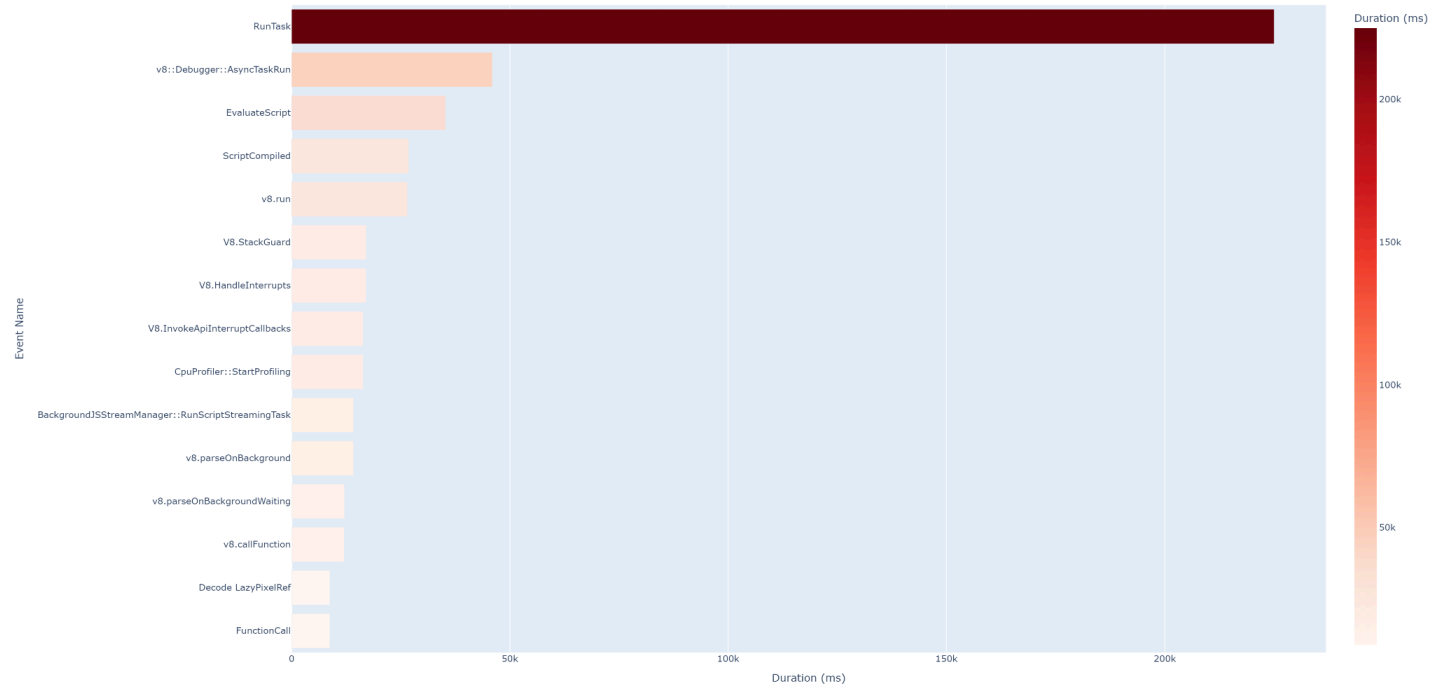
Five resources are marked as render-blocking, preventing the first paint for nearly 1 second.

- Critical CSS & Fonts: The main stylesheet index-B1X112b2.css and several Google Font stylesheets (s-2575, s-2580, s-2579) must be fully downloaded before any content appears.
- Third-Party Impact: Stripe's controller script (stripe.js) and execution (e) contribute 83 ms of main thread time during the load

Total Execution Time Breakdown by Page



Top 15 Most Expensive Tasks (Total Time Across All Pages)



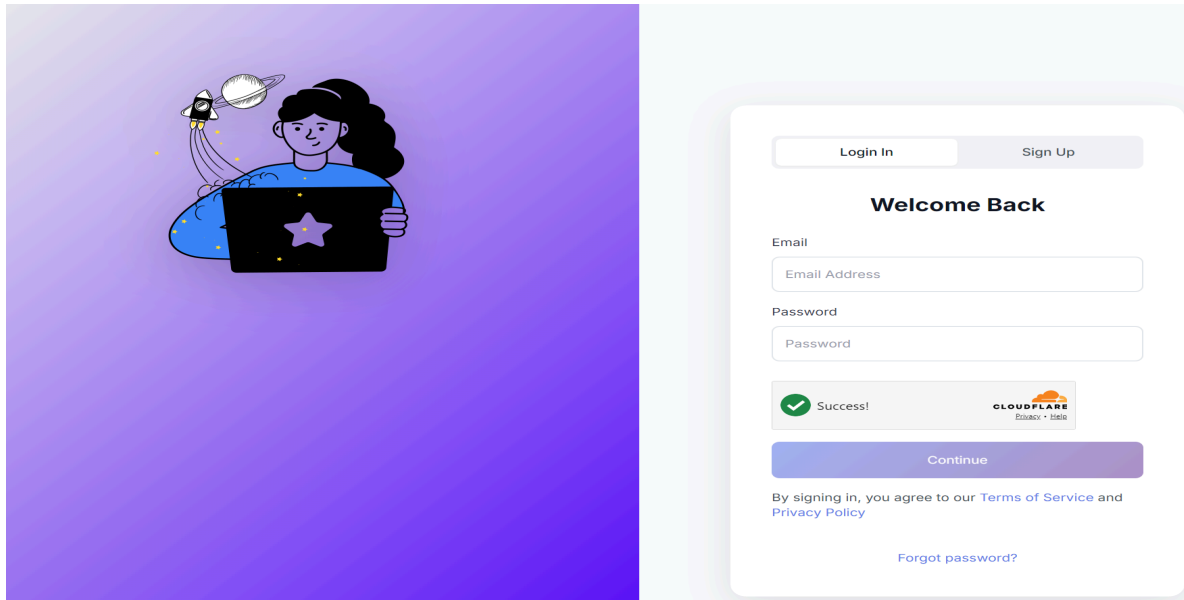
Comparative Performance Metrics Summary

All reported pages are served via Cloudflare.

Page File	Total Load Time (s)	Connection Time (ms)	TTFB (ms)	Image Decode (ms)	Total UI Frozen (s)	Longest Single UI Freeze (s)
sat.meghdo.com/tutorials	19325.17	1608.83	9139.40	1824.66	34.34	3.33
sat.meghdo.com/(Home)	19425.53	1386.79	9498.90	695.87	29.83	1.94
sat.meghdo.com/teach-and-earn	19503.81	573.52	7756.85	431.59	25.58	1.59
sat.meghdo.com/student-benefits	19558.81	616.27	9099.52	472.73	32.83	2.01
sat.meghdo.com/search-result	19598.72	793.26	9212.77	972.81	30.89	1.90
sat.meghdo.com/about	19641.38	369.75	7836.31	417.58	27.14	1.88
sat.meghdo.com/contact	19742.98	2022.25	7868.42	998.93	50.32	3.11
sat.meghdo.com/dashboard	19779.18	487.98	6872.42	424.54	39.09	3.20

2] pl.meghdo.com:

Page 1: <https://pl.meghdo.com/login>



The LCP element is an image (Round 3-11-CsLA-ilP.png).

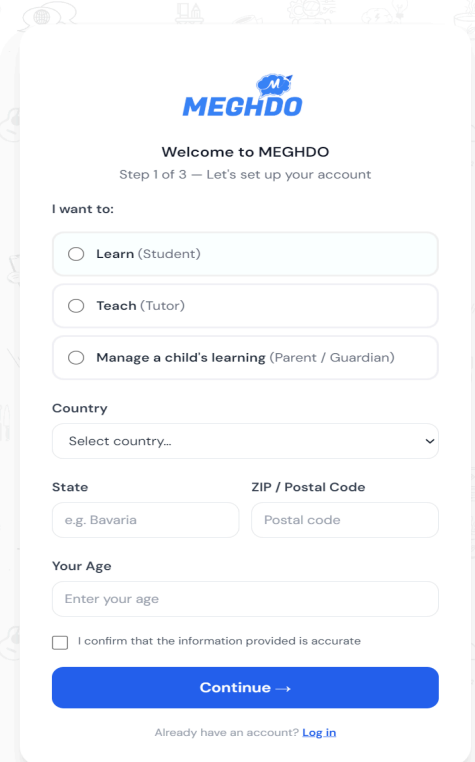
- Late Discovery: The image is initiated by vendor-eVk5PToZ.js. Because the browser has to download, parse, and execute this JavaScript before it even knows the image exists, the download doesn't start until 324 ms into the page load.
- Fast Transfer: The actual download duration was only 1 ms, indicating that the network itself is not the bottleneck.
- Render Blocking: After the image finishes, it waits for the main thread to complete a 24 ms Layout (Layout) and significant JavaScript execution before it finally paints at 458 ms.

Main Thread Bottlenecks

The main thread is active with several tasks that contribute to the render delay:

- JavaScript Execution: A 113 ms long task (RunTask) is dominated by evaluating index-CngBkwzo.js. Within this, an anonymous function (ProfileCall) at line 19 spends 19 ms calling Intl.DateTimeFormat(). Synchronously creating internationalization objects during the critical path is an avoidable expense.
- React Reconciliation: The ls and bs functions in vendor-eVk5PToZ.js spend approximately 53 ms processing the component tree and managing application state.
- Extension Overhead: The extension "Allow Copy - Select & Enable Right Click" (contentScript.js) contributes 36 ms of execution time during the loading window, which adds to the overall congestion.

Page 2] <https://pl.meghdo.com/onboarding>:

The image shows a mobile app onboarding screen for 'MEGHDO'. The background is a light gray with a dense pattern of small, faint icons related to education and technology. The app interface is a white card with rounded corners. At the top is the 'MEGHDO' logo in blue. Below it, the text 'Welcome to MEGHDO' and 'Step 1 of 3 — Let's set up your account' are displayed. The form asks 'I want to:' and provides three radio button options: 'Learn (Student)', 'Teach (Tutor)', and 'Manage a child's learning (Parent / Guardian)'. Below these are input fields for 'Country' (a dropdown menu), 'State' (with 'e.g. Bavaria' as a placeholder), and 'ZIP / Postal Code' (with 'Postal code' as a placeholder). There is also a 'Your Age' input field. A checkbox with the text 'I confirm that the information provided is accurate' is present. At the bottom of the form is a large blue 'Continue' button with a right-pointing arrow. Below the button, the text 'Already have an account? [Log in](#)' is visible.

The LCP element is an image (Meghdo Logo_Final.png). Its timing is broken down as follows:

- Resource Load Delay (636 ms): The image request was not initiated until 642 ms into the page load. It was discovered by vendor-evk5PToZ.js, which is a late-discovered dependency.
- Resource Load Duration (4 ms): The actual network transfer was extremely fast.
- Element Render Delay (815 ms): After the image finished downloading, it took another 815 ms to actually render. This delay was caused by heavy main thread activity, including a 388 ms long task (RunTask) and significant layout/style calculations.

The main thread is heavily congested during the critical loading window:

- Long Tasks: There are several significant tasks, most notably a 388 ms task (RunTask) that blocks the engine from processing the LCP render.
- Layout & Style (135 ms): Combined, layout and style recalculations consume a large portion of the early execution time, particularly a 57 ms layout (Layout) triggered shortly after the image download.
- Third-Party Impact: An extension "Allow Copy - Select & Enable Right Click" is contributing roughly 99 ms of execution time, which can interfere with page performance in a real-world user environment.

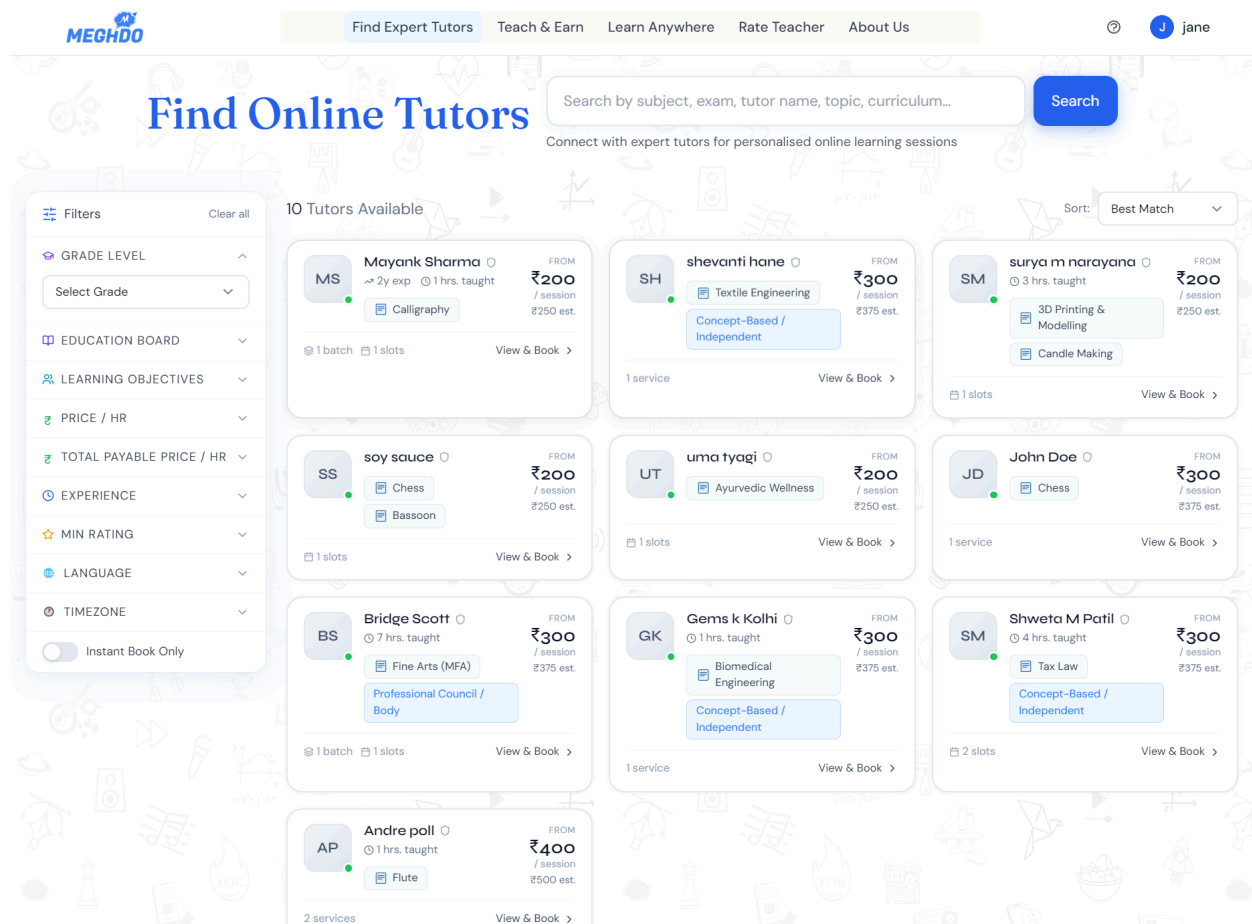


The LCP element is a background image (Bg_Landing-Page-k8NwtUI8.jpg). Its timing is categorized into four phases:

- Resource Load Delay (732 ms): The image was not requested until 736 ms into the page load. While it is discoverable in the initial document, it lacks `fetchpriority="high"`, which prevents the browser from prioritizing it early enough.
- Resource Load Duration (349 ms): The network transfer was relatively fast, but it only began after the load delay.
- Element Render Delay (2,156 ms): This is the most critical bottleneck, accounting for over 66% of the total LCP time. After the image was downloaded at 1,086 ms, it took another 2.1 seconds to paint.

The extreme render delay is caused by heavy main thread activity that blocks the browser from painting the downloaded image:

- Long Tasks and Layout: There are multiple long tasks immediately following the image download. A series of Layout (Layout) events totaling 312 ms and Recalculate Style (Layout) events block the rendering path.
- JavaScript Execution: Total task time on the main thread exceeds 1,200 ms. Significant time is spent in `index-CngBkwzo.js` and vendor scripts, which keep the CPU busy while the LCP element is waiting to be painted.
- Third-Party Impact: The extension "Allow Copy - Select & Enable Right Click" contributed 103 ms of main thread time. Additionally, scripts from Stripe and Cloudflare add nearly 100 ms of execution overhead during the critical loading window.



The LCP is triggered by a text element, which means there is no image download to wait for. Instead, the delay is entirely due to the browser's inability to paint the content until the main thread becomes free and the critical CSS/JS is processed.

- **Render-Blocking Overhead:** The initial render is blocked by multiple high-priority requests, including CSS files `index-B1X112b2.css` and multiple Google Fonts stylesheets.
- **Main Thread Congestion:** A series of Long Tasks (RunTask) immediately follow the page load, preventing the browser from performing the final paint.

Main Thread Bottlenecks

The main thread is busy for nearly the entire first second of the page load:

- **Layout Engine (151 ms):** A major Layout (Layout) event occurs at 773 ms, triggered by JavaScript execution. This layout cost is likely exacerbated by a large DOM size.
- **Heavy JavaScript Execution:**
- **A 261 ms task (RunTask)** is spent evaluating `index-CngBkwzo.js`. Within this script, a specific function (at line 19, column 966977) spends 47 ms just calling `Intl.DateTimeFormat()`, which is a surprisingly expensive operation for the critical path.
- **React Reconciliation:** The function `bs` in `vendor-eVkJ5PTToZ.js` is responsible for a significant amount of work (totaling 176 ms) as it processes the component tree.
- **Extension Interference:** An extension, "Allow Copy - Select & Enable Right Click", injected a content script `contentScript.js` that added execution overhead during the initial loading window.



Find Expert TutorsTeach & EarnLearn AnywhereRate TeacherAbout Us

⌵ jane

Teach what you know. Build your reputation. Earn globally.

Teach students across India and beyond through structured one-on-one sessions, batches, workshops, and live learning experiences.

[Start Teaching on MEGHDO](#)[Need help deciding what to teach?](#)

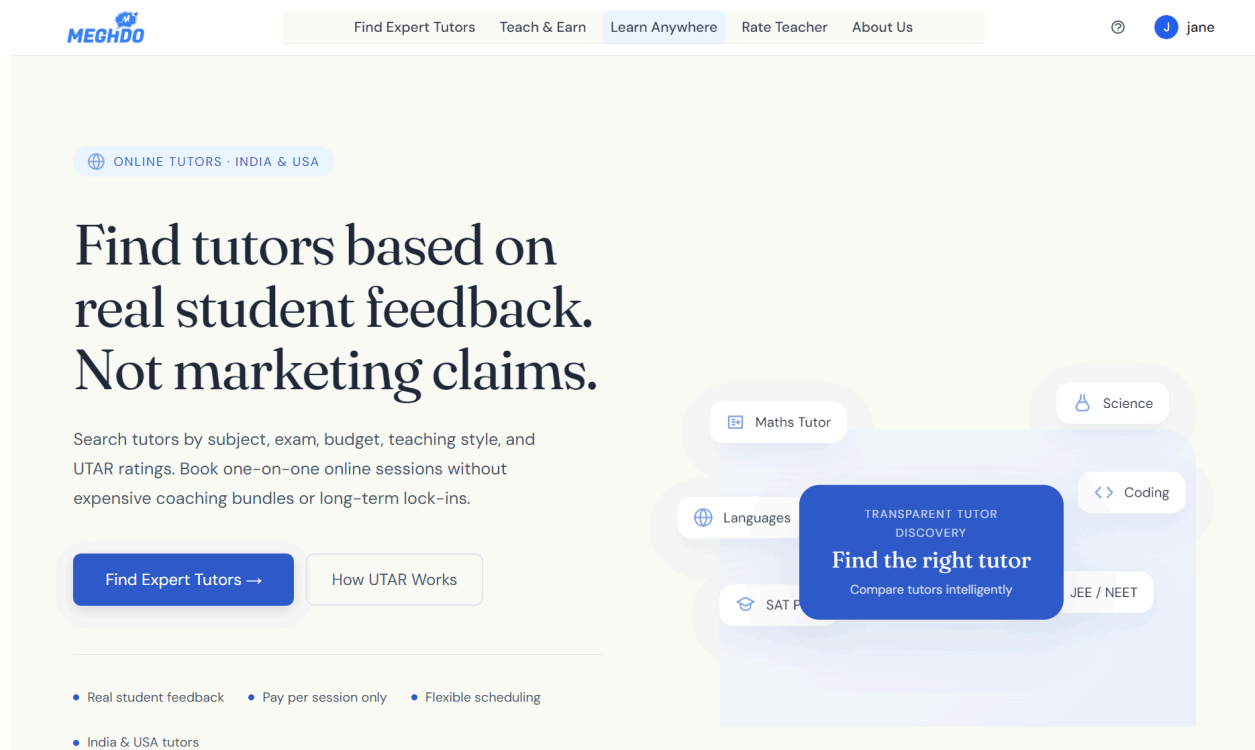
Why educators choose MEGHDO

Flexible teaching Teach part-time or full-time on your own schedule.	Global students Reach learners across India, the United States and other countries.	Transparent earnings Set your own session pricing and receive clear payouts.	Professional reputation Build your UTAR reputation through structured student feedback.
--	---	--	---

Main Thread Bottlenecks

The main thread is heavily congested for nearly the entire duration of the page load, primarily due to large JavaScript bundles and complex initialization:

- **Long Tasks:** There are several significant tasks blocking the engine, most notably a 236 ms task (RunTask) and a 159 ms task (RunTask).
- **JavaScript Execution:**
- **Expensive Initialization:** In `index-CngBkwzo.js`, a significant amount of time is spent in anonymous functions related to data processing or internationalization. For example, a 40 ms function (ProfileCall) is spent processing large arrays or objects during script evaluation.
- **React Reconciliation:** The script `vendor-eVv5PTToZ.js` spends considerable time in functions like `bs` (76 ms) and `xi`, which handle the rendering and update logic of the application.
- **Layout Engine (136 ms):** A massive Layout (Layout) event occurs at 714 ms. This single event blocks the LCP paint and is likely triggered by the JavaScript execution finishing and demanding a visual update of the DOM.
- **Render-Blocking Resources:** The page wait for several CSS files to load, including `index-B1X112b2.css` and Google Fonts, before it can even begin the first paint.



Main Thread Bottlenecks

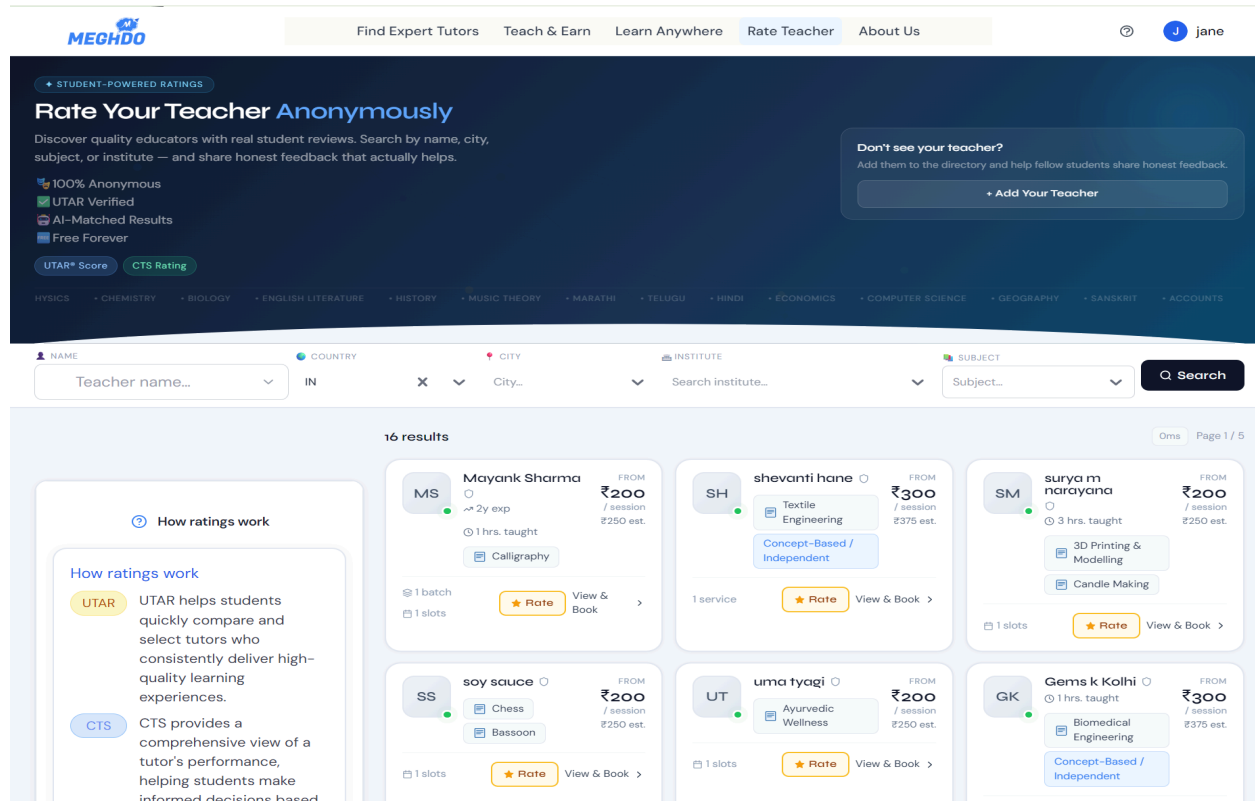
The main thread is heavily congested for the first 1.5 seconds, preventing the initial paint:

- **Massive Layout Operation (460 ms):** A single Layout (Layout) event takes nearly half a second. This is a significant bottleneck that blocks the LCP render and is likely triggered by complex DOM manipulations during application hydration.
- **Heavy JavaScript Execution:**
- **Data Processing in index-CngBkwzo.js:** A 261 ms task (RunTask) is spent evaluating and initializing the main application script. Specifically, a function at index-CngBkwzo.js:59966 (referenced within anonymous code (ProfileCall)) spends significant time iterating over large objects for what appears to be timezone or internationalization setup.
- **React Hydration:** The script vendor-eVkJ5PToZ.js spends a combined 154 ms in the z and ls functions, which are part of the React rendering and reconciliation process.
- **Extension Overhead:** The extension "Allow Copy - Select & Enable Right Click" (contentScript.js) contributes 124 ms of main thread time during the critical loading window, further delaying the page's ability to render.

Network & Render-Blocking Resources

The page's initial render is also gated by several render-blocking requests:

- index-B1X112b2.css and env-config.js must finish loading before the browser can begin painting.
- Multiple Google Fonts requests are made early on. While they use display=swap, the sheer number of font stylesheets can add to the initial network contention.



Main Thread & Rendering Bottlenecks

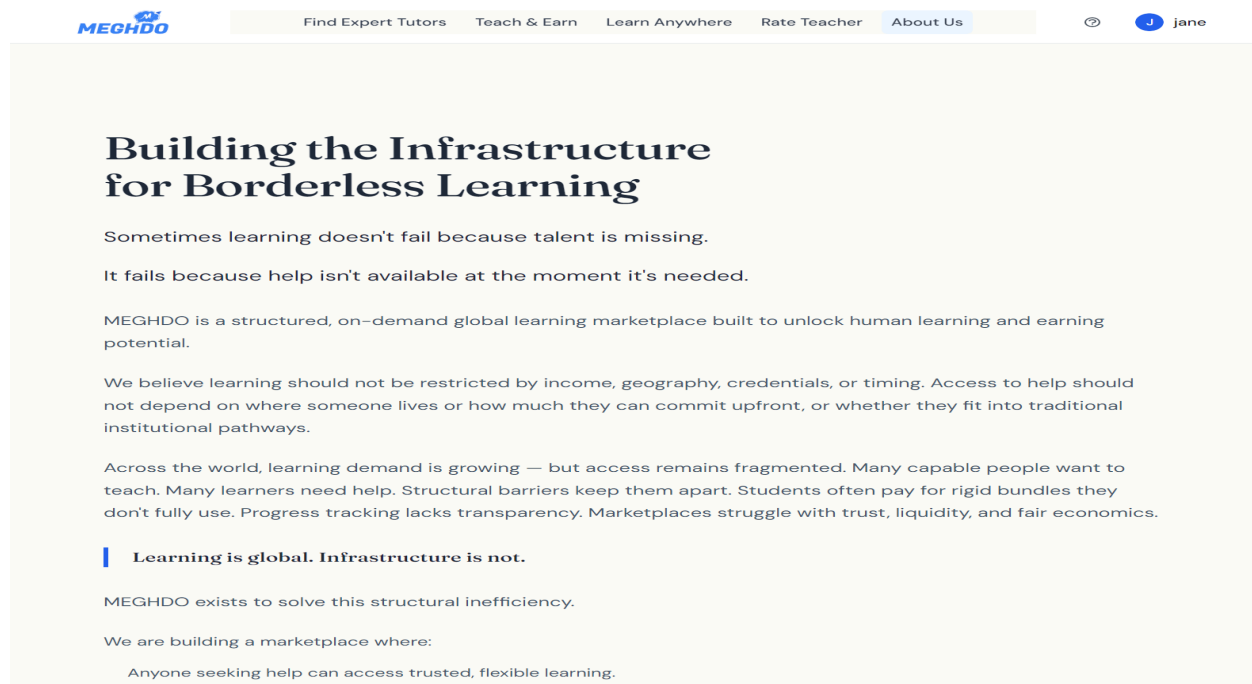
The main thread is heavily congested, leading to several "Long Tasks" that block rendering:

- **Excessive Layout Work (771 ms):** The browser spends a total of 771 ms performing layout. One major Layout (Layout) event alone takes 364 ms. This indicates a very complex DOM where any change requires the browser to recalculate the position of hundreds of elements.
- **JavaScript Execution Bottlenecks:**
- **React Hydration:** A significant amount of time is spent in vendor-eVvk5PToZ.js. The function bs (ProfileCall) (part of React's reconciliation engine) accounts for 1,049 ms of total time. This suggests that the application is performing a very heavy "hydration" process or rendering a massive component tree all at once.
- **Expensive Initialization:** In index-CngBkwzo.js, an anonymous function (ProfileCall) at line 19 is responsible for 60 ms of synchronous work, likely processing data or setting up application state.
- **Third-Party Impact:**
 - **Stripe:** Scripts from <https://js.stripe.com> contribute 235 ms to main thread activity.
 - **Extension Overhead:** The extension "Allow Copy - Select & Enable Right Click" (contentScript.js) adds 137 ms of execution time, which can interfere with the page's initial responsiveness.

DOM Size Issues

The page suffers from an excessive DOM size, which directly correlates with the long layout times:

- Style recalculations affected up to 615 elements at once.
- Layout updates frequently required processing every single node in the tree (e.g., 775 nodes).



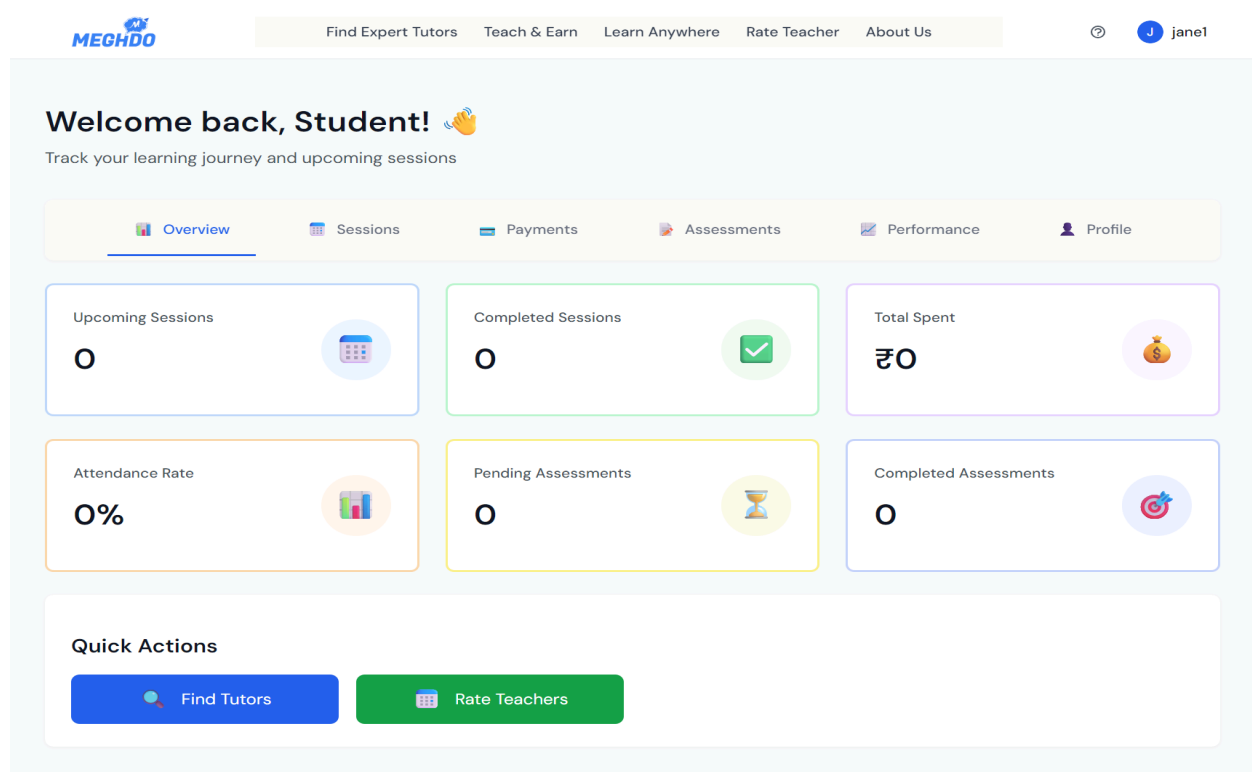
Main Thread & Rendering Bottlenecks

The main thread is heavily congested for nearly the entire first three seconds of the page load.

- **Excessive Layout Work (438 ms total):** A major Layout (Layout) event takes 289 ms, followed by another Layout (Layout) event taking 132 ms. These synchronous operations block the browser from painting the LCP text. Large layouts of this scale usually indicate a very complex DOM being modified during application hydration.
- **Heavy JavaScript Execution:**
- **React Hydration:** The script vendor-eVkJ5PToZ.js is responsible for significant work. The functions ls (154 ms) and bs (React's reconciliation engine) are processing a large component tree, preventing the initial paint.
- **Data Processing in index-CngBkwzo.js:** A 261 ms long task (RunTask) is spent evaluating the main application script. Within this, multiple anonymous functions are iterating over large objects (like timezone/location data), adding significant overhead. One specific anonymous function (ProfileCall) at line 19 spends 36 ms on synchronous processing.
- **Third-Party and Extension Impact:**
- **Stripe:** The script stripe.js contributes 117 ms to main thread activity during the critical loading window.
- **Extension Overhead:** The extension "Allow Copy - Select & Enable Right Click" (contentScript.js) adds 92 ms of execution time, further delaying the page's ability to render.
- **Network and Dependencies**

The page is blocked by several render-blocking resources that must finish before any content can appear:

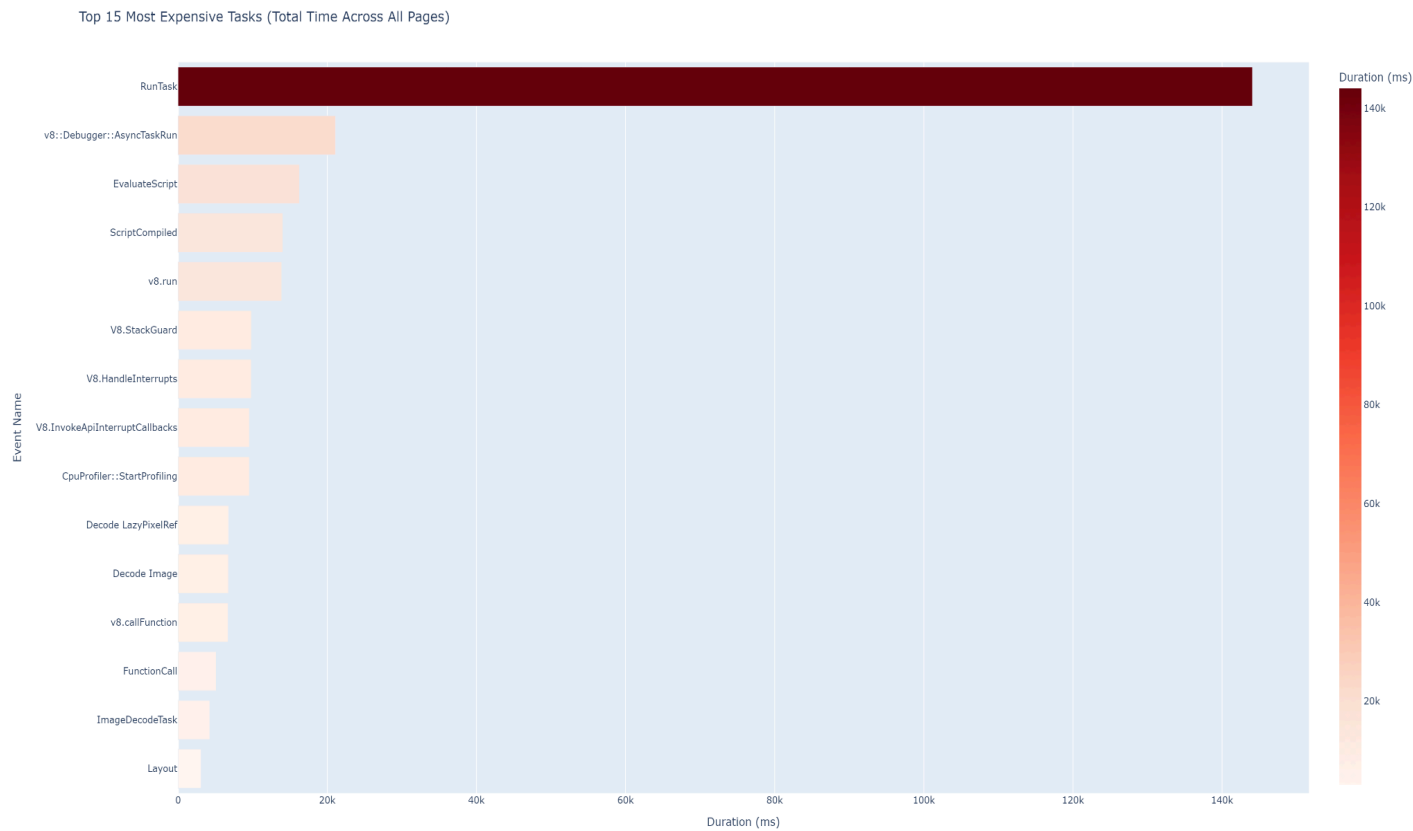
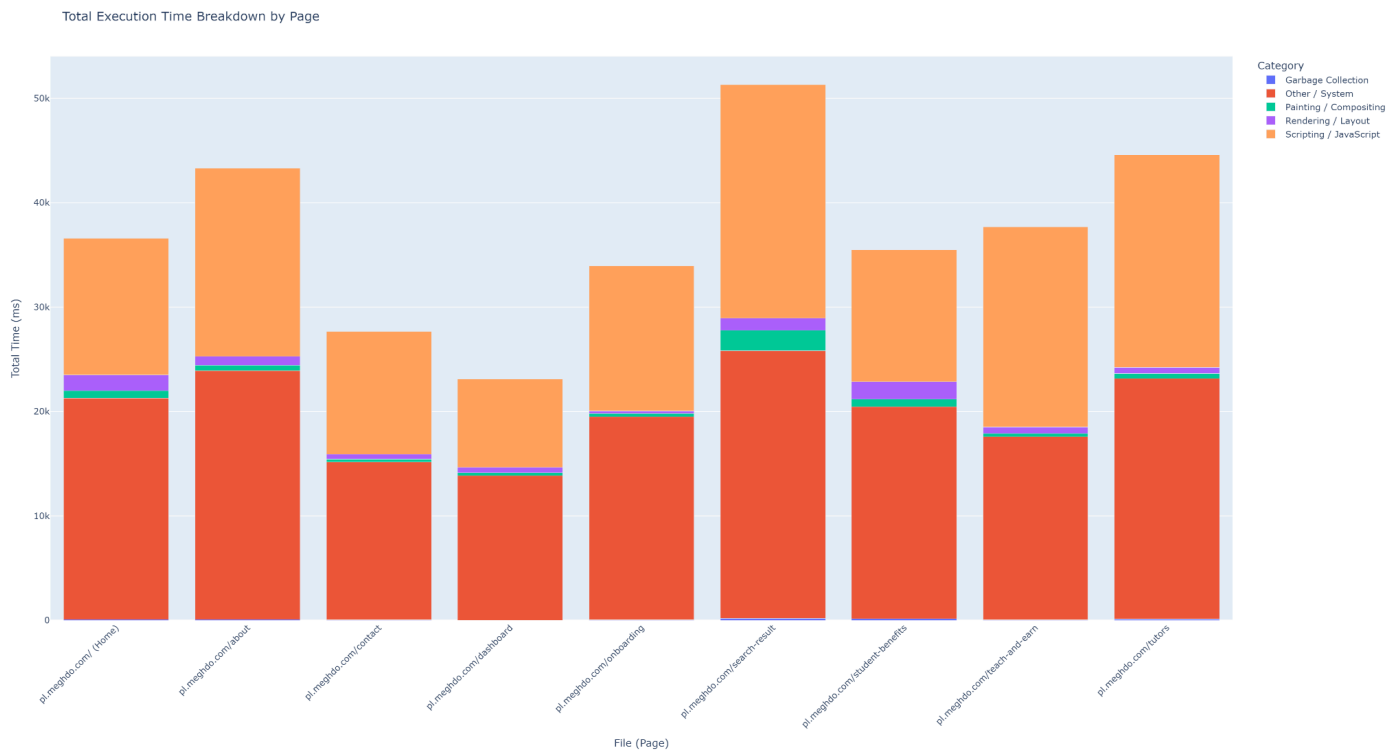
- index-B1X112b2.css and env-config.js are critical dependencies.
- The page loads multiple Google Fonts stylesheets, which, while using font-display: swap, still contribute to the initial connection and parsing overhead.



Main Thread & Rendering Bottlenecks

The main thread is heavily congested for the first 1.5 seconds, preventing the initial paint:

- Excessive Layout Work (259 ms total): A major Layout (Layout) event takes 125 ms, blocking the LCP render. This operation is triggered by the application's hydration process and indicates a complex DOM tree being modified all at once.
- Heavy JavaScript Execution:
- Expensive Initialization: In index-CngBkwzo.js, a 277 ms task (RunTask) is spent evaluating the script. A specific anonymous function (ProfileCall) spends 41 ms just calling Intl.DateTimeFormat(). Creating these internationalization objects synchronously during the critical path is a significant bottleneck.
- React Hydration: The script vendor-eVvk5PToZ.js spends a combined 138 ms in the ls and bs functions, which handle React's reconciliation and rendering logic.
- Third-Party & Extension Overhead:
- Stripe: Scripts from js.stripe.com contribute 111 ms of main thread time during the loading window.
- Extension Interference: The extension "Allow Copy - Select & Enable Right Click" (contentScript.js) adds 84 ms of execution time, further delaying the page's ability to yield to the paint engine.



Comparative Performance Metrics Summary

(pl.meghdo.com)

All reported pages are served via **Cloudflare**.

Page File	Total Load Time (s)	Connection Time (ms)	TTFB (ms)	Image Decode (ms)	Total UI Frozen (s)	Longest Single UI Freeze (s)
pl.meghdo.com/about	19212.15	8285.18	8671.92	397.20	15.80	1.26
pl.meghdo.com/dashb oard	18997.08	2143.76	4215.17	389.13	6.23	0.46
pl.meghdo.com/tutors	19106.38	0.00	7433.24	1478.08	16.63	1.49
pl.meghdo.com/conta ct	19142.87	282.56	4143.52	575.88	9.38	0.72
pl.meghdo.com/ (Home)	18768.58	0.00	3266.31	677.08	10.49	0.86
pl.meghdo.com/stude nt-benefits	19247.36	365.15	3497.88	428.92	10.70	0.65
pl.meghdo.com/onboa rding	18861.14	2002.79	3352.24	1472.26	11.00	0.84
pl.meghdo.com/searc h-result	19053.13	0.00	7216.99	928.78	18.67	1.50
pl.meghdo.com/teach- and-earn	19286.36	0.00	2804.68	398.23	16.66	1.48

Comparison between both <https://pl.meghdo.com/>
And
<https://sat.meghdo.com/>

Metric (Averages)	sat.meghdo.com	pl.meghdo.com	Delta (%)
Total Load Time	19,572s	18,964s	-3.1%
TTFB (Latency)	8,411ms	4,956ms	-41.1%
Total UI Frozen	33.75s	12.84s	-62.0%

Detailed Analysis and Performance Conclusions

The comparison between the *sat* and *pl* subdomains reveals a systemic performance disparity. While both domains suffer from significant Total Load Times—averaging nearly 20 seconds—the **pl.meghdo.com** environment demonstrates markedly superior efficiency in server responsiveness (TTFB) and main-thread availability.

1. Latency and Connectivity: The *pl* domain's TTFB is approximately 41.1% faster than *sat*. This suggests more optimal server-side processing or closer edge-node proximity within the Cloudflare network for the *pl* assets.

2. Main Thread Congestion: The most staggering difference is found in the "Total UI Frozen" metric. The *sat* domain averages 33.75 seconds of total UI freeze time across its pages, compared to 12.84 seconds on *pl*, a reduction of 62.0%. This indicates that the JavaScript bundles on *sat* (notably `index-CZbBWt_M.js`) contain significantly more expensive reconciliation logic or heavier third-party initialization than the equivalent bundles on *pl*.

3. Final Conclusion: Users on the *sat* subdomain experience nearly triple the interactivity delay of those on the *pl* subdomain. Optimization efforts should prioritize reducing the script evaluation overhead and optimizing the React hydration process specifically for the *sat* application core.